

DSY1107 · EVALUACION PARCIAL N.º 2

Pedidos360

Arquitectura cloud-native multi-nube
con OAuth 2.0 y OpenID Connect

ESTUDIANTES

Ismael Oyarzun Montiel
Diego Villota

IDENTIDAD

Microsoft Entra ID

COMPUTO

Amazon Web Services

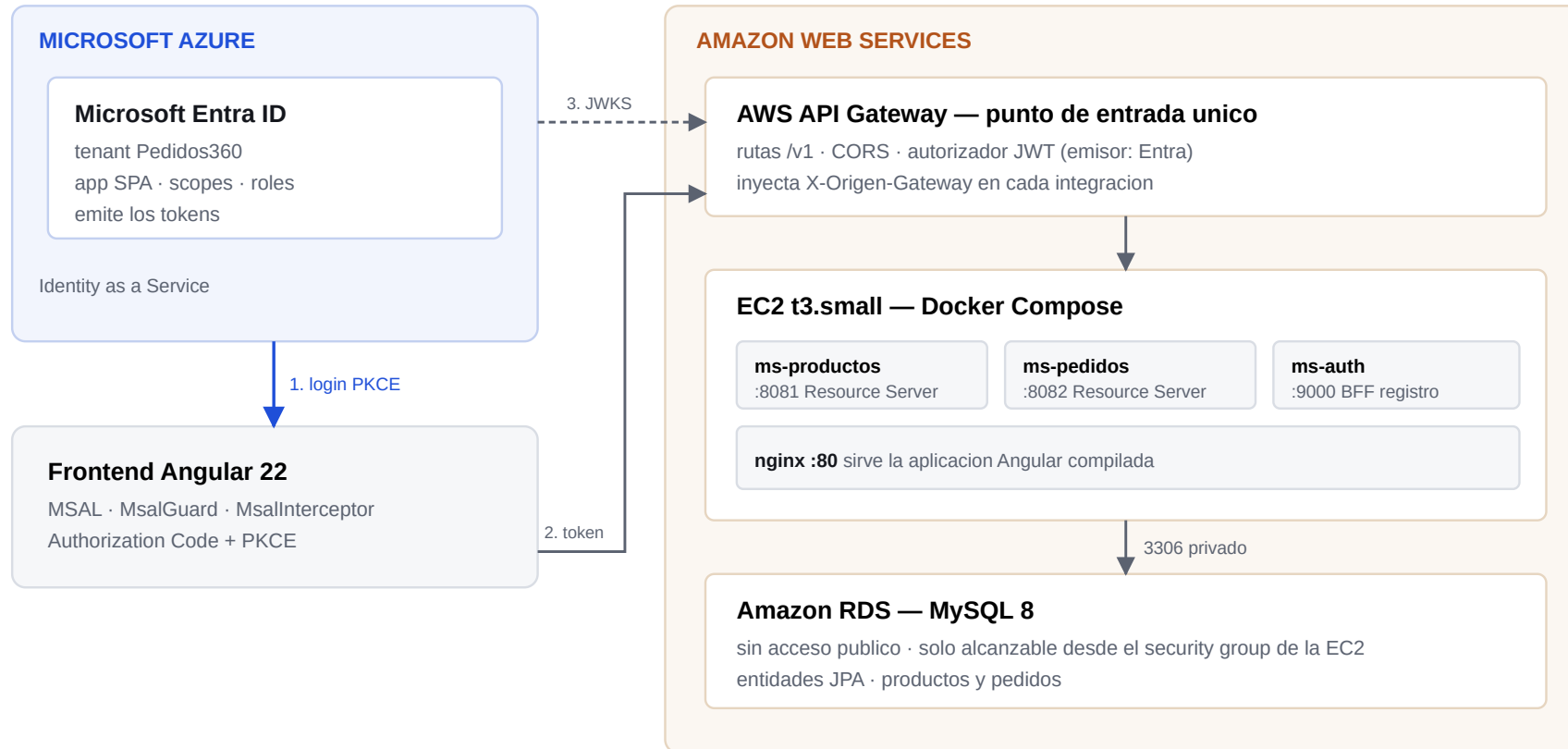
Dos nubes que no se conocen

La identidad la administra **Azure**. El compute vive en **AWS**. No comparten red, ni base de datos, ni credenciales.

El unico puente es el **JWKS**: una URL publica desde la que AWS descarga las llaves con que verificar las firmas que emite Azure.

Solo criptografia de llave publica.

ARQUITECTURA



COMPONENTES

Tres microservicios y un frontend

COMPONENTE	ROL
front-angular	Angular 22 + MSAL. Authorization Code con PKCE
ms-productos	Catalogo. Resource Server OAuth 2.0
ms-pedidos	Pedidos. Autorizacion por rol y por scope
ms-auth	BFF: registra usuarios en el tenant via Microsoft Graph
Amazon RDS	MySQL 8, sin acceso desde Internet

DEMOSTRACION

El tenant en el IDaaS

- Tenant **Pedidos360** en Microsoft Entra ID
- Usuarios registrados con sus **roles** asignados
- Uno de ellos creado desde el propio frontend

Entra ID Free

No trae autoservicio de registro. El formulario del frontend llama a `ms-auth`, que crea la cuenta con Microsoft Graph usando su propio secreto. Una SPA no podría: tendría que exponerlo.

DEMOSTRACION

La aplicacion registrada

- Redirecciones de tipo **SPA**, con `Web` vacio
- Scopes `productos.leer` y `pedidos.escribir`
- Roles `ADMIN` y `USER`

La decision que no perdona

Con `Web`, Azure exige un `client_secret` que una SPA no puede esconder. Con `SPA` habilita PKCE. Elegir mal no da ningun error evidente: el login falla recien al canjear el codigo.

DEMOSTRACION

Inicio de sesion y token

Login con Microsoft desde la aplicacion, y en la pagina **Mi token**: el `id_token` con la identidad, el `access_token` con `scp` y `roles` , y lo que ve el microservicio tras validar la firma.

Un JWT va **firmado, no cifrado**. Cualquiera puede leer su contenido; nadie puede modificarlo sin invalidar la firma.

Que el login funcione no demuestra PKCE

Un flujo implicito tambien iniciaria sesion. Lo que lo demuestra es la peticion al endpoint de autorizacion:

```
response_type      = code
code_challenge     = wKgFDWE2Xsjbnw0Vb50XLGrTVfQYtl9D9VRU-B1J_qk
code_challenge_method = S256
```

El navegador inventa un `code_verifier`, envia solo su hash, y presenta el original al canjear el codigo. Un codigo interceptado no sirve sin el verifier.

DEMOSTRACION

El API Manager

- La instancia creada y en funcionamiento
- **Rutas limpias** con la version en el path publico
- Una **integracion** por ruta hacia cada microservicio
- **Autorizador JWT** apuntado al JWKS de Azure

```
GET /v1/productos  
→ :8081/api/v1/productos
```

```
POST /v1/pedidos  
→ :8082/api/v1/pedidos
```

La version vive en la ruta publica: publicar una `v2` no afecta a los clientes de `v1`.

DEMOSTRACION

CORS, sin sobrepermisos

ORIGEN DE LA PETICION

RESPUESTA DEL PREFLIGHT

El dominio del frontend

permitido

Cualquier otro origen

bloqueado

Origenes declarados de forma explicita, no un comodin, y solo con los metodos y cabeceras que la aplicacion necesita.

DEMOSTRACION

200, 401 y 403

SITUACION	CODIGO	QUIEN RESPONDE
Token valido y permisos suficientes	200	el microservicio
Sin token, o token invalido	401	autorizador del API Gateway
Token valido, sin el rol o el scope	403	Spring Security
Peticion que no viene del gateway	403	filtro de origen

401 significa *no se quien eres*. **403** significa *se quien eres y no puedes*.

El gateway no oculta la instancia

La EC2 tiene IP publica. Un `curl` directo se saltaria el gateway y su autorizador por completo.

El gateway inyecta una cabecera secreta en cada integracion; los servicios rechazan con **403** lo que llegue sin ella.

Lo que esta medida no es

No es aislamiento de red. Quien conozca el secreto puede llamar directo. Lo correcto seria subred privada con VPC Link: unos USD 16 al mes, fuera del presupuesto de la cuenta academica.

Desplegado, activo e integrado

EC2

Instancia corriendo con cuatro contenedores: los tres microservicios y nginx sirviendo la aplicacion.

RDS

MySQL disponible y **sin acceso publico**: solo lo alcanza el security group de la EC2.

API Gateway

Stage con HTTPS, unico punto de entrada desde Internet.

Lo que aprendi construyendolo

- Validar por **emisor** y no contra un IdP fijo: migrar a Entra ID fue cambiar dos variables de entorno, sin tocar una linea de Java
- Construi un Identity Provider propio para entender el protocolo, y despues **lo elimine**: dejarlo habria sido un camino de acceso paralelo y mas debil
- La seguridad se verifica, no se declara: **20 de 20** comprobaciones contra el despliegue y **9 de 9** del flujo de inicio de sesion

Guion, minuto a minuto

0:00	Laminas 1-3. El problema y la arquitectura. Frase de apertura: "la identidad la administra Azure, el computo esta en AWS, y el unico puente es el JWKS".
1:00	Laminas 4-5 · Azure. Overview del tenant, Users con sus roles, App registration: Authentication (SPA, Web vacio), Expose an API, App roles.
2:30	Laminas 6-7 · Login en vivo. Iniciar sesion, ir a "Mi token", mostrar scp y roles. Luego la lamina de PKCE con code_challenge_method=S256.
4:00	Laminas 8-9 · AWS. API Gateway: Routes, Integrations, Authorization, CORS.
5:30	Lamina 10 · Codigos. Thunder Client o docs/evidencias/08-codigos-http.txt: 200, 401, 403 por rol y 403 por acceso directo.
6:45	Laminas 11-12. El punto de entrada unico con su limitacion, y las consolas de EC2 y RDS.
7:45	Lamina 13. Cierre.

Preguntas probables

JWT doble	El gateway protege el borde, no el servidor: la EC2 tiene IP publica. Por eso ademas exigimos la cabecera del gateway.
¿Es seguro?	Lo correcto seria VPC Link con subred privada, ~USD 16/mes, fuera del presupuesto. La cabecera es una mitigacion consciente, no aislamiento de red.
External ID	Imposible con estas cuentas: ciamDirectories y Azure for Students no comparten ninguna region, y la restriccion es una politica de sistema que ni un Owner puede eximir.
IdP propio	Lo construi para entender el protocolo y sirvio de andamio antes de que existiera el tenant. Luego migre a Entra y lo elimine.
Minimo privilegio	ms-auth tiene AppRoleAssignment.ReadWrite.All, mas poder del necesario. Graph no ofrece uno mas acotado para asignar roles de la propia app.